

Signals

A Signal is a wrapper around a value that **knows when it changes**, and can tell Angular exactly who is using it — so only the UI that actually depends on it updates.

1 · WHY SIGNALS EXIST

Before Signals: Angular had to guess

A plain `count = 0` is just a number — nothing reactive about it. When it changes, Angular doesn't automatically know *that specific* value changed. Historically Angular used **Zone.js** to patch async APIs (clicks, timers, HTTP) and, on any of them firing, walked the **entire** component tree checking every binding — even ones that didn't change.

Signals fix this: they track their own readers, so Angular updates **only** what actually depends on them.

2 · SIGNAL() BASICS

`count` vs. `count()` — do not mix these up

`count` — the Signal itself

```
count = signal(0);

console.log(count);
// -> Signal function reference,
// NOT the number
```

`count()` — the current value

```
console.log(count());
// -> 0

count.set(10);
count.update(v => v + 1);
```

In the template we write `{{ count() }}`, not `{{ count }}` — calling it also registers the template as a dependent, so it updates automatically later.

3 · COMPUTED()

Derived, read-only state

```
count = signal(10);
doubleCount = computed(() => count() * 2);

console.log(doubleCount()); // 20
count.set(20);
console.log(doubleCount()); // 40 — recalculated automatically
```

`computed()` tracks any Signal read inside its function and re-runs only when those change. It's **read-only** — no `.set()`. Use it for totals, filtered lists, formatted strings, validation flags.

4 · EFFECT()

Running side effects when Signals change

```
effect(() => {
  localStorage.setItem('cart-count', String(cartCount()));
});
```

Same dependency tracking as `computed()`, but for side effects (logging, localStorage, a non-Angular widget) — it returns nothing usable. Don't use it to derive a value you'll use elsewhere; that's what `computed()` is for.

5 · SIGNALS + HTTPCLIENT

A realistic component: loading, error, success

```
export class ProductsComponent {
  products = signal<Product[]>([]);
  isLoading = signal(false);
  error = signal('');
  private productService = inject(ProductService);

  ngOnInit() {
    this.isLoading.set(true);
    this.productService.getProducts().subscribe({
      next: (res) => { this.products.set(res.products); this.isLoading.set(false); },
      error: () => { this.error.set('Failed to load.');
```

Service

HttpClient



API

Observable



Signal

.set(...)



UI

@for products()

6 · SIGNALS VS. RXJS

Not a replacement — a complement

SIGNALS — GREAT FOR STATE	OBSERVABLES (RXJS) — GREAT FOR STREAMS
Local / component state	Async event streams (HTTP, WebSockets)
Derived values via <code>computed()</code>	Complex operator pipelines (debounce, switchMap)
Simple, synchronous reads with <code>count()</code>	Multiple values over time, cancellation

```
import { toSignal, toObservable } from '@angular/core/rxjs-interop';

// Observable -> Signal
products = toSignal(this.productService.getProducts(), { initialValue: [] });

// Signal -> Observable
search$ = toObservable(this.search);
```

HttpClient still returns Observables — that isn't changing. Convert at the boundary with `toSignal` / `toObservable`.

